

url video

keywords name
date author

= metadata

CLOUD COMPUTING: STORAGE & DATA

Module 5: Storage and Data Management

load

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases



Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases

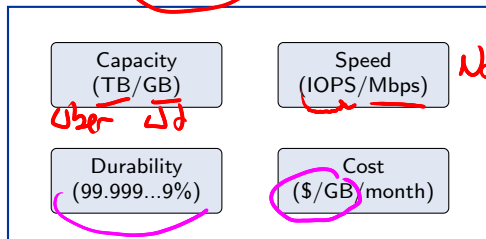


What Does "Storage" Mean?

Key Properties

Every storage system trades off four properties: capacity, speed, durability, and cost.

Pick 2-3, optimize for your workload



2 hours



video → streaming → storage • *link to cloud*
 ↳ transcript → "Acta": *comprovisos*



Storage Types: Block, File, Object

Block Storage

Raw disk volumes
Fast, structured

VM disks, Databases

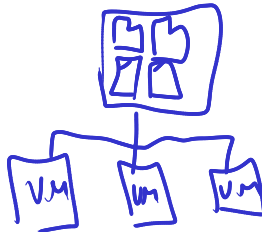
snowflake



File Storage

Shared filesystem
Concurrent access

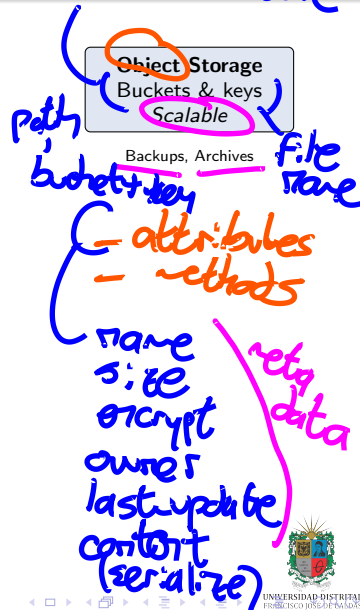
NFS SMB shares



Object Storage

Buckets & keys
Scalable

Backups, Archives



Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases



Block Storage

severs VM

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- Fast: low latency, high IOPS
- Attached: must be bound to a compute instance
- Limited: per compute instance storage capacity
- HDD/HUD options trade cost vs. speed
- Snapshots, cloning, backup

Use Cases

Database storage, transactional workloads, application data requiring low latency.

Block Storage

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- Fast: low latency, high IOPS
- Attached: must be bound to a compute instance
- Durable: survives instance shutdown (persistent)
- SSD/HDD options: trade cost vs. speed
- Snapshots: point-in-time backups

Use Cases

Database storage, transactional workloads, application data requiring low latency.

Block Storage

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

Use Cases

Database storage, transactional workloads, application data requiring low latency.

Block Storage

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

Use Cases

Database storage, transactional workloads, application data requiring low latency.

Block Storage

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

\$\$\$ = low

Use Cases

Database storage, transactional workloads, application data requiring low latency.

Block Storage

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

Use Cases

Database storage, transactional workloads, application data requiring low latency.

Block Storage

Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

Use Cases

Database storage, transactional workloads, application data requiring low latency.

File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- Shared: multiple VMs can access simultaneously
- Managed: no filesystem management needed



Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- **Shared:** multiple VMs read/write simultaneously
- Managed: no filesystem management needed
- Scalable: grows automatically with demand
- Locking: prevents concurrent corruption
- Networking: accessed over network (slightly slower)

Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- **Shared:** multiple VMs read/write simultaneously
- **Managed:** no filesystem management needed
- Scalable: grows automatically with demand
- Locking: prevents concurrent corruption
- Networking: accessed over network (slightly slower)

Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand

• **Locking**: prevents concurrent corruption

• **Networking**: accessed over network (distributed)

hor: zonta
scaling

Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



File Storage

Definition

Shared filesystem (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

Use Cases

Shared team storage, media files, machine learning datasets, shared caches.

GitHub



Object Storage

Definition

Unlimited scalability storage accessed via APIs (HTTP REST). Data organized as objects (key-value pairs) in buckets.

Characteristics:

bucket (red handwritten) /
Folder /
key (red handwritten) - file-text (blue handwritten)

Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- Unlimited capacity: petabytes to exabytes
- Durable: 11 9's (99.999999999%) durability
- No structure: flat namespace, any data type
- Versioning: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable** 11 9's (99.999999999%) durability
- No structure: flat namespace, any data type
- Versioning: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- Versioning: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- **Lifecycle policies**: auto-archive or delete old data
- **Cheap**: lowest cost per GB

after 5 years
go to vaulty
chest

Use Cases

Backups, archives, data lake, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- **Lifecycle policies**: auto-archive or delete old data
- **Cheap**: lowest cost per GB

Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

Object Storage

Definition

Unlimited scalability storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- **Lifecycle policies**: auto-archive or delete old data
- **Cheap**: lowest cost per GB

Use Cases

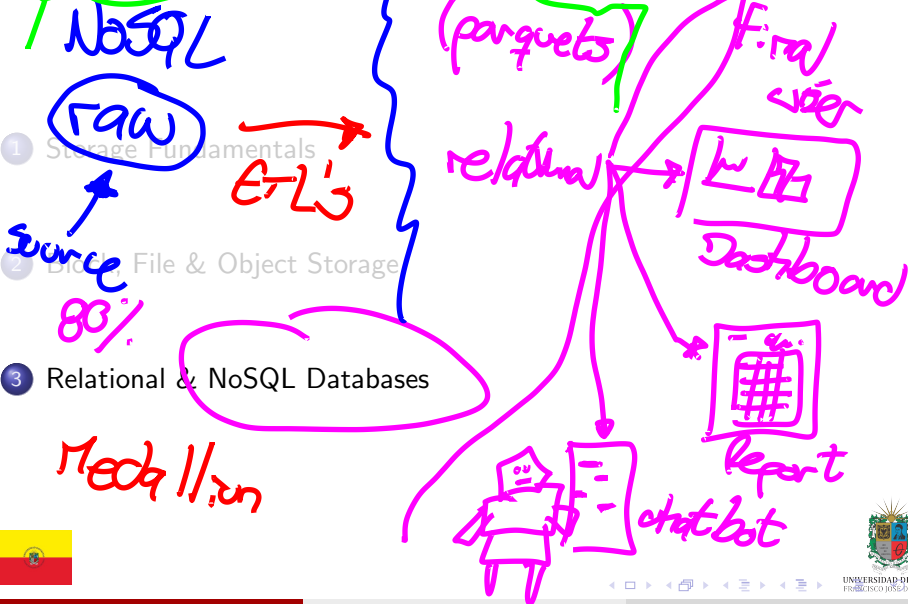
Backups, archives, data lakes, static website hosting, media distribution.

Outline

Data Lake

Offline

ML



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*) **ACID** transactions guarantee data consistency.

Key Concepts:

Atomicity
Consistency
Isolation (Concurrency)
Durability

When to Use

Structured data, complex queries, transactions (banking, ecommerce).



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

Key Concepts:

- **Tables:** rows and columns with schema
- Primary key: unique identifier per row
- Foreign key: link to another table
- ACID: Atomicity, Consistency, Isolation, Durability
- Indexes: speed up queries on specific columns

metadata

query

When to Use

Structured data, complex queries, transactions (banking, ecommerce).



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- Foreign key: link to another table
- ACID: Atomicity, Consistency, Isolation, Durability
- Indexes: speed up queries on specific columns

When to Use

Structured data, complex queries, transactions (banking, ecommerce).



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- ACID: Atomicity, Consistency, Isolation, Durability
- Indexes: speed up queries on specific columns

When to Use

Structured data, complex queries, transactions (banking, ecommerce).



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- **ACID**: Atomicity, Consistency, Isolation, Durability
- **Indexes**: speed up queries on specific columns

When to Use

Structured data, complex queries, transactions (banking, ecommerce).



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- **ACID**: Atomicity, Consistency, Isolation, Durability
- **Indexes**: speed up queries on specific columns

Handwritten table with pink border:

id	name	email
1	~	blw
2	-	teow
3	-	a@od

Handwritten table with blue border:

email	name
a@od	2(3)
blw	0(1)
teow	1(2)

When to Use

Structured data, complex queries, consistency (banking)

slow / 9th



Relational Databases

The Model

Structured data in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- **ACID**: Atomicity, Consistency, Isolation, Durability
- **Indexes**: speed up queries on specific columns

When to Use

Structured data, complex queries, transactions (banking, ecommerce).

Good!



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide column).

Time to live
key1: value,
key2: value?,
key3: value
REDIS

MONGODB



→ dir / Map / JSON

→ collection

→ document

Trade-off: CAP Theorem

Consistency, Availability, Partition tolerance.



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

Types:

- **Key-Value:** Redis — ultra-fast caching
- **Document:** MongoDB — flexible JSON documents
- **Graph:** Neo4j — relationships and patterns
- **Wide-Column:** Cassandra — massive distributed data
- **Search:** ElasticSearch — full-text search

background

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

Types:

- **Key-Value:** Redis — ultra-fast caching
- **Document:** MongoDB — flexible JSON documents
- **Graph:** Neo4j — relationships and patterns
- **Wide-Column:** Cassandra — massive distributed data
- **Search:** Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.

Facebook



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: ~~ElasticSearch~~ — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



NoSQL Databases

Beyond Relational

Flexible schemas, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases

data cloud
migration

all cloud
providers



Recap: Module 5

- **Block Storage:** attached disk volumes, fast, structured access
- **File Storage:** shared filesystem, concurrent access, NFS/SMB
- **Object Storage:** unlimited scalability, durable, cheap (S3, MinIO)
- **Relational:** structured, ACID, complex queries, strong consistency
- **NoSQL:** flexible, scalable, optimized for specific patterns
- **Decision:** choose storage type based on access pattern, scale, consistency needs

→ File User

raw



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

